DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

Aurora swallows a duck, and Postgres tries life without VACUUM

Issue #9 | Tue, 13 Oct 2026 | ⏱️ ~44 min read | Level 3/5

🗄️ Today's two stories

1. **Aurora PostgreSQL can now query Parquet and Iceberg files in S3 directly, and the engine doing the work is DuckDB, embedded inside the database.** It's the first product to come out of AWS buying DuckLabs, the company behind DuckDB, five weeks earlier. A Postgres developer advocate has since proven it from the query plans, and a Postgres consultant has mapped where it silently falls back to slow paths. We explain foreign tables, the two kinds of "pushdown", how memory and caching are shared with your OLTP workload, and the one `char(25)` column that can ruin a join.
2. **OrioleDB, a replacement storage engine for Postgres that needs no VACUUM, is now in public beta on Supabase's paid plans.** It's been five years in the making, and it rethinks almost everything under the `CREATE TABLE`: rows live inside the primary-key index, old versions go to an undo log instead of piling up in the table, and the shared-buffer hash table is gone. Supabase claims up to 1.8x the throughput of Postgres's normal heap, but there's no point-in-time recovery yet and no SLA. We explain why Postgres needs VACUUM in the first place, and what OrioleDB does instead.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 Aurora PostgreSQL gets a DuckDB inside

AMAZON AURORA POSTGRESQL · AURORA_ANALYTICS · ANNOUNCED 30 SEP 2026 · PLAN TEARDOWNS 5 AND 8 OCT · ~20 MIN READ

For twenty years, the advice has been the same: don't run analytics on your production database. Copy the data out, every night or every few minutes, into a warehouse built for big scans, and query it there. On 30 September AWS shipped a feature that quietly argues the opposite. Your Aurora PostgreSQL database can now read years of history straight out of Parquet files in S3 and join it with today's rows in one SQL statement. The engine reading those files isn't Postgres at all. It's a small, fast, open-source analytics database from Amsterdam that AWS bought in August, now living inside the Postgres server process.

WHAT HAPPENED

On **30 September 2026**, the AWS News Blog announced that Amazon Aurora PostgreSQL "now supports direct querying of Apache Iceberg and Parquet data in your data lake". The post, by Esra Kayabali, says plainly that "DuckDB is now embedded directly within Aurora PostgreSQL", and adds that DuckLabs, "the team behind DuckDB", recently joined Amazon.[1]

The facts, from the announcement and the Aurora user guide:

- The feature is an extension called `aurora_analytics`. You turn it on with a cluster parameter, attach an IAM role, and run `CREATE EXTENSION aurora_analytics;`.[1,3]
- It needs **Aurora PostgreSQL 17.11 or higher, or 18.6 or higher**, "including Aurora Serverless v2".[2]
- It reads **Parquet** files and **Iceberg** tables in Amazon S3, Amazon S3 Tables, or the AWS Glue Data Catalog. Other Iceberg catalogs work through Glue's catalog federation.[2]
- It's **read-only**: no `INSERT`, `UPDATE`, `DELETE` or `COPY FROM` into the lake. You can copy lake data *into* Aurora with `CREATE TABLE AS SELECT`, `INSERT INTO ... SELECT` or `MERGE INTO`.[4,2]
- It's available in all commercial AWS Regions and the GovCloud (US) Regions, and there's "no additional charge". You pay for the extra Aurora compute your queries burn and for the S3 requests they make.[1]

AWS published no benchmark numbers in the announcement.[1]

Two independent write-ups followed within days. On **5 October** (edited 10 October), Franck Pachot, a developer advocate for PostgreSQL on Azure at Microsoft, set out to "*prove*" the DuckDB claim "rather than take it on faith", comparing Aurora's query plans with those of the open-source `pg_duckdb` extension. His verdict: "They are the same plan."[7] And on **8 October**, Christophe Pettus (PGX Inc.) compared `aurora_analytics` with three other ways of querying a lake from Postgres, concentrating on the question that decides performance: "when the query runs, which planner is in charge of it?"[8]

THE BACKGROUND YOU NEED

Two shapes of data

Postgres is a **row store** *≈ a database that stores each row's columns together on disk, ideal for fetching or updating whole records one at a time*. Each table is a set of pages holding whole rows, which is exactly what you want when an API call reads one customer or updates one order. It's what you don't want for "total revenue per region over five years", which needs two columns out of twenty, across a billion rows. A row store has to drag all twenty columns through memory to get at the two.

Analytical engines flip the layout. A **column store** *≈ a database or file format that stores each column's values together, so a query reads only the columns it needs and compresses them well* keeps each column's values together, so the revenue query reads just the bytes it needs. They also tend to execute queries **vectorized** *≈ processing a batch of values (say, a thousand rows of one column) per operation instead of one row at a time, which uses modern CPUs far more efficiently*, working on thousands of values per step instead of one row at a time.

That split is why the industry calls them **OLTP** *≈ online transaction processing: many small, concurrent reads and writes of individual records, like an app's checkout flow* and **OLAP** *≈ online analytical processing: fewer, much larger queries that scan and aggregate lots of data, like reports and dashboards* workloads, and why they've usually lived in separate systems.

The data lake on S3

The cheapest place to keep years of history is object storage, and the de facto file format there is **Parquet** *≈ an open columnar file format; each file is split into row groups, and each column chunk carries min/max statistics so readers can skip*

data that can't match a filter. A Parquet file is split into chunks of rows, and for each chunk it records summary statistics, such as the minimum and maximum value of each column. A reader looking for `order_date >= '2024-01-01'` can skip a whole chunk whose maximum date is in 2022 without reading it.

A pile of Parquet files isn't a table, though. You need to know which files make up the current version, what the schema is, and how to change it safely. **Apache Iceberg** *≈ an open table format: metadata files on object storage that track which data files form each version (snapshot) of a table, enabling schema changes, time travel and safe concurrent writes* adds that layer, and a **catalog** *≈ a service that maps table names to their current Iceberg metadata, such as AWS Glue Data Catalog or any Iceberg REST catalog* tells engines where each table's metadata lives.

So the usual architecture has been: your app writes to Postgres; a pipeline exports changes to S3 as Parquet or Iceberg; a warehouse or query engine (Athena, Redshift, Snowflake, Spark) analyses it. The AWS post pitches `aurora_analytics` as removing the trip back: you can join live rows with the lake "without ETL".[1]

Postgres's escape hatch: foreign tables

Postgres has had a way to query data that isn't stored in Postgres for years. The docs explain that it "implements portions of the SQL/MED specification, allowing you to access data that resides outside PostgreSQL using regular SQL queries". A **foreign data wrapper** *≈ a Postgres extension (FDW) that knows how to talk to an external data source and fetch rows from it on demand* is "a library that can communicate with an external data source". You create a **foreign server** *≈ a Postgres object describing one external data source and how to reach it, used by a foreign data wrapper*, then **foreign tables** *≈ tables defined in Postgres whose data lives elsewhere; queries against them are passed to a foreign data wrapper*, which "can be used in queries just like a normal table, but … ha[ve] no storage in the PostgreSQL server".[9]

Extensions can also take over parts of query execution through Postgres's **CustomScan** *≈ a Postgres extension API that lets an extension replace part of the executor with its own scan or join implementation, shown as a "Custom Scan" node in EXPLAIN* interface, which "allow[s] extensions to override a part of executor's scanning behavior by providing custom implementations".[10] Keep that in mind: it's how a whole foreign analytics engine can appear as a single node in your `EXPLAIN` output.

DuckDB, and DuckDB inside Postgres

DuckDB *≈ an open-source (MIT-licensed), in-process columnar analytical database: it runs as a library inside your program, like SQLite, but for analytics* is often described as "SQLite for analytics": a library that runs inside your process, columnar and vectorized, and very good at reading Parquet. It's created by Hannes Mühleisen and Mark Raasveldt, and maintained under the non-profit DuckDB Foundation with an MIT licence.[11]

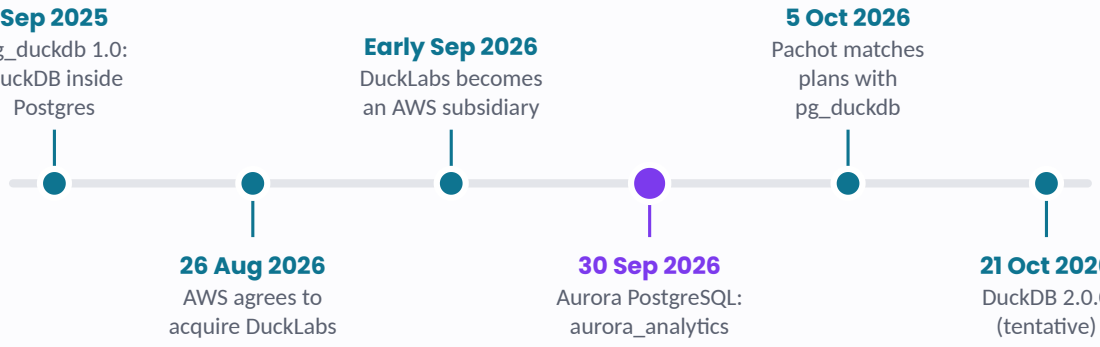
DuckDB has been plugged into Postgres before. In September 2025, MotherDuck announced **pg_duckdb 1.0**, "an open-source PostgreSQL extension that brings DuckDB's vectorized analytical engine directly inside PostgreSQL". It "embeds a DuckDB instance directly into your existing PostgreSQL process", can read Parquet on S3 (plus CSV, JSON, Iceberg and Delta), and with `duckdb.force_execution = true` can even run queries on ordinary Postgres tables through DuckDB's engine.[12] It's available on self-managed Postgres and, as Pachot notes, on Azure Database for PostgreSQL.[7]

Amazon buys the duck

On **26 August 2026**, AWS announced a definitive agreement to acquire DuckLabs, the Amsterdam company DuckDB's creators founded "a little more than five years" earlier, which had grown to more than 30 people without venture capital.[13] DuckDB's own announcement said DuckLabs would become an AWS subsidiary in early September, and that DuckDB, DuckLake and the other extensions stay MIT-licensed under the Foundation, with "no changes for our projects' roadmap, licensing, and governance model".[11] AWS said it planned to combine DuckDB with Redshift, Athena, EMR, Glue and SageMaker, and claimed its Amazon Quick service already runs DuckDB in its query engine, with "over 2.5B queries" and a 30% average latency reduction (AWS's numbers).[14] Aurora wasn't on that list. Five weeks later, it was the first to ship.

MotherDuck's CEO Jordan Tigani, whose company is built on DuckDB, gave the sceptic's reading at the time: "That's Amazon's playbook, after all: wait until an open source project gets big enough, then launch it as a service."[13]

From in-process analytics to inside Aurora



Sources: MotherDuck, DuckDB, AWS News Blog, Pachot, DuckDB release calendar.

HOW IT WORKS

Setting it up

The tutorial in the Aurora user guide walks through the whole thing. In short: create an IAM role that `rds.amazonaws.com` can assume, with `s3:ListBucket`, `s3:GetObject` and `s3:GetBucketLocation` on your bucket; set `aurora_analytics.enabled` to `true` in a cluster parameter group; attach the role to the cluster with the feature name `AuroraAnalytics`; and, if the cluster is in a private subnet, add an S3 gateway VPC endpoint. Then, from `psql`, you create the extension and a foreign table, as below.[3]

The `aurora_analytics_server` foreign server is created for you by the extension, and "All foreign tables use this server". The `location` can be an S3 URI, a Glue table ARN or an S3 Tables ARN. For Iceberg you can pin a `snapshot` ID or a `timestamp` to query an older version of the table (the two are mutually exclusive, and a timestamp is resolved when the foreign table is created). If you list columns yourself, "only those columns are projected during query execution".[4] For whole Glue databases, `IMPORT FOREIGN SCHEMA` creates foreign tables in bulk.[1]

```
CREATE EXTENSION aurora_analytics;
CREATE FOREIGN TABLE ft_orders ()  -- () = infer columns
SERVER aurora_analytics_server
OPTIONS (
    location 's3://your-data-bucket/data/orders/',
    format 'parquet',
    region 'us-east-1'
);
```

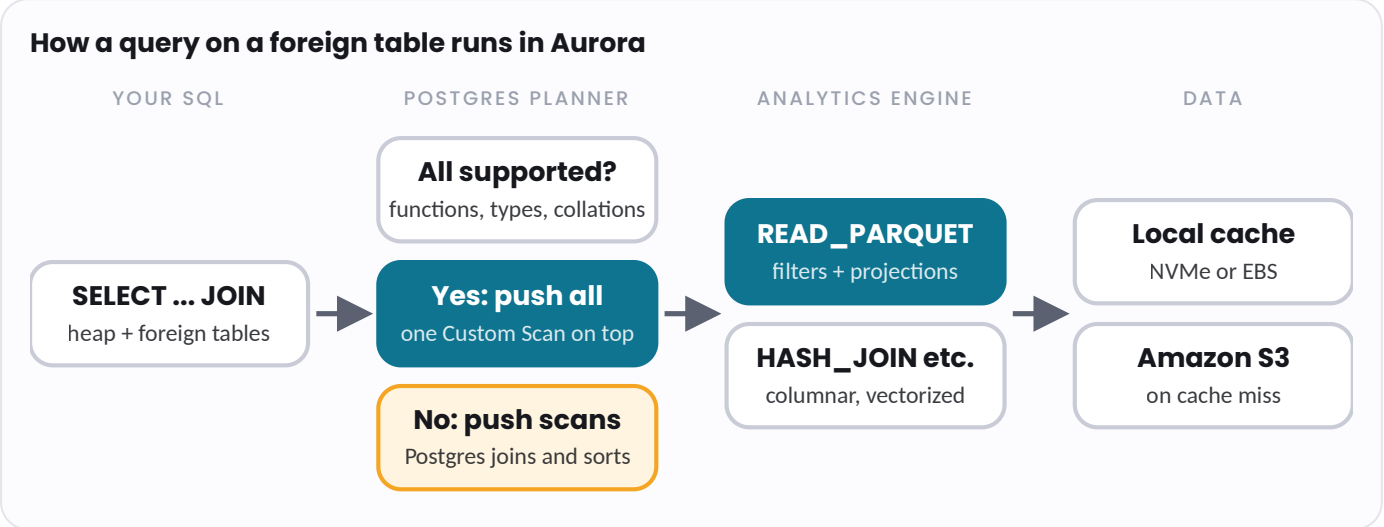
The announcement's example shows the point of it all: one `UNION ALL` that returns a customer's seven most recent transactions from a normal Aurora table and the rest from a five-year Parquet history in S3.[1]

Two ways a query can run

Here's where it gets interesting. The execution-plan page of the Aurora guide describes two strategies, "chosen automatically by the planner".[5]

Full query pushdown. "The entire query (including JOINs, aggregations, subqueries, CTEs, ORDER BY, and LIMIT) is pushed to the analytics engine. This is the fastest execution path." In `EXPLAIN` you see a single `Custom Scan` node at the top, with the analytics engine's own operators underneath.[5]

Table-scan pushdown (the fallback). When the query contains something the engine can't handle, Aurora pushes down only the scan of each foreign table, still with filters and column pruning, "The JOINs, sorts, and unsupported operations then run in PostgreSQL." You see `Custom Scan` nodes nested inside ordinary Postgres nodes like `Hash Join` and `HashAggregate`.[5]



The guide's worked examples use TPC-H at scale factor 100 (about 34 GB of Parquet) on a `db.r8gd.8xlarge`. In its version of TPC-H Query 1, the `READ_PARQUET` node scanned 21 files, pushed the `l_shipdate` filter into the scan, read only 4 of lineitem's 16 columns, and reports "Rows Removed by Filter: 15303257125", which the guide attributes to "min/max statistics". With a warm cache, the plan shows about 4.5 GB served from the local cache and 16 kB read remotely, with just two S3 GET requests.[5]

In the four-table join of Query 5, the plan shows something more sophisticated: **dynamic filters** ≈ *filters created during execution: after building a join's hash table, the engine passes the key range and a Bloom filter to the other side's scan so it can skip rows that can't match*. The line `l_orderkey IN BF(o_orderkey)` means that once the engine knows which orders survive the date filter, it builds a **Bloom filter** ≈ *a compact probabilistic set: it can say "definitely not present" or "possibly present", so it's a cheap way to discard non-matching keys early* of their keys and uses it to skip lineitem rows while scanning.[5] That run was only partly cached: about 3.3 GB from cache, about 2.5 GB from S3, and 4,922 S3 GET requests.[5]

Joining your live tables

The part AWS's announcement emphasises is the hybrid join. A local heap table can be pulled *into* the analytics engine: in the guide's Example 3, the plan contains a `POSTGRES_SCAN` node reading the local table inside the pushed-down `HASH_JOIN`, so the whole join and aggregation run in the columnar engine.[5]

But that only works "as long as the query uses only supported functions, data types, and collations". In the guide's very next example, the local `nation` table has an `n_name` column of type `character(25)`. That type isn't supported for pushdown, so the plan falls back: Postgres does a `Seq Scan` on `nation`, a `Hash Join`, and a `HashAggregate`, and the engine only scans lineitem. `EXPLAIN VERBOSE` explains why, under "Unsupported Pushdown Expressions": `Type ... character(25)`.[5]

The guide lists the categories that cause fallback:[5]

- **Functions** such as `soundex()`, `uuid_generate_v4()`, PostGIS functions and your own UDFs.
- **Types** such as `vector`, `integer[]`, `geometry` and `bytea` casts.
- **Operators** such as `~` (regex), `LIKE` under a non-C collation, and pgvector's `<+>`.
- **Collations**: any non-C collation in a comparison. The fix is an explicit `COLLATE "C"`.
- **Aggregates** such as `percent_rank()` and `array_agg()`.
- **Syntax** such as the `ONLY` keyword, recursive CTEs and dynamic data masking.

Same intent, different plan

Falls back: non-C collation

```
SELECT count(*)
FROM ft_customers
WHERE name LIKE 'Smith%';
-- EXPLAIN VERBOSE lists an
-- Unsupported Pushdown
-- Expression (Operator)
```

Full pushdown: explicit C

```
SELECT count(*)
FROM ft_customers
WHERE name COLLATE "C"
      LIKE 'Smith%';
-- one Custom Scan at the top
```

Pattern from the Aurora guide's tuning section; the table name is illustrative, and note the C collation compares bytes, not your locale's rules.

Pettus's write-up turns this into a warning: "one `char(n)` column in a dimension table moves your join from DuckDB's planner to PostgreSQL's, silently". The danger of the fallback isn't just that Postgres does the join. Postgres's planner relies on table statistics that Parquet files don't give it, and the docs confirm that `SET`

`STATISTICS` "ha[s] no effect" on these foreign tables.[4] A planner working from a default guess, Pettus writes, "will cheerfully put a 133-million-row scan on the inner side of a nested loop".[8]

How we know it's DuckDB

AWS says it's DuckDB. Pachot wanted evidence. He ran the same query over the same Parquet file on Aurora (Serverless v2, engine 17.11) and on open-source pg_duckdb (the `pgduckdb/pgduckdb:17-v1.1.1` Docker image, with `duckdb.force_execution = true`). Both plans contained the same operator names: `READ_PARQUET`, `PERFECT_HASH_GROUP_BY`, `count_star()`, `ORDER_BY`. The `Pushdown SQL` line in Aurora's plan calls `system.main.read_parquet(...)`, and the user guide's own plans do too.[7,5]

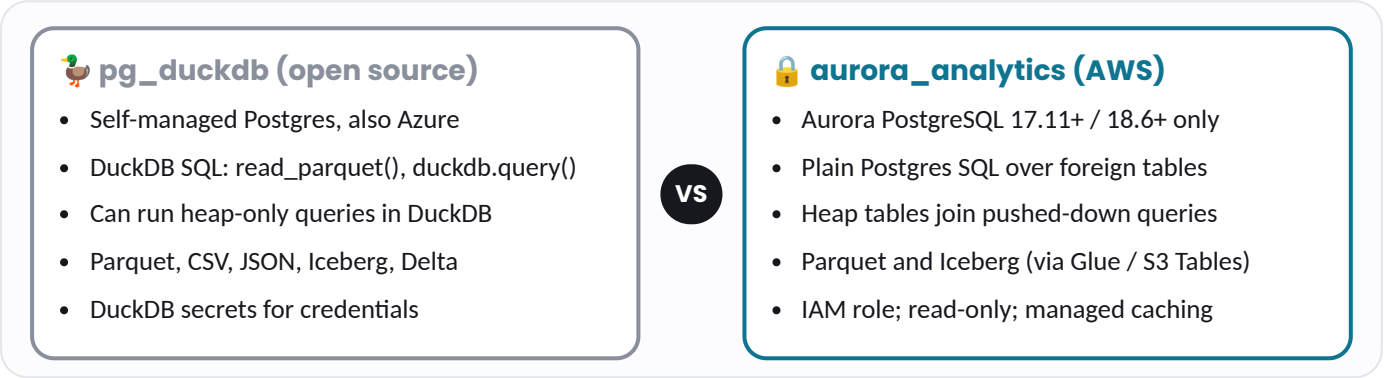
His strongest clue is an internal compression operator that appears in both plans: `__internal_compress_integral_utinyint(#0, 1920)`. He reads 1920 as the minimum `production_year` in the file, which DuckDB subtracts so the column fits in a single byte during grouping. An independent engine, he argues, wouldn't reproduce DuckDB's internal names *and* pick the same base value.[7]

Be clear about what that does and doesn't prove. It's one query on one roughly 500,000-row file, the Aurora run was served from cache, no DuckDB version was reported for either side, and he works for a competing cloud provider that sells pg_duckdb. He concludes Aurora "uses DuckDB or DuckDB-derived components under its Foreign Data Wrapper".[7] Given that AWS itself says DuckDB, that's a reasonable conclusion. The more useful part of his post is what Aurora *doesn't* let you do.

A sealed duck

Pachot calls it DuckDB "sealed behind an FDW". Compared with pg_duckdb, he found:[7]

- No direct DuckDB SQL. `read_parquet()`, `QUALIFY`, `approx_count_distinct`, list and struct literals and `::HUGEINT` are rejected; there's no `duckdb.query()` and no access to DuckDB extensions.
- No acceleration of local Postgres tables on their own. (They can join a pushed-down query, as above, but you can't send a heap-only query to DuckDB the way `duckdb.force_execution` does.)
- Read-only: no `COPY ... TO` the lake. Iceberg only through Glue, and no Delta Lake.
- Credentials come from the IAM role, not DuckDB secrets.
- The extension surface is tiny: he counts 366 objects in pg_duckdb and four in `aurora_analytics` (the FDW, its handler and validator, and one server).

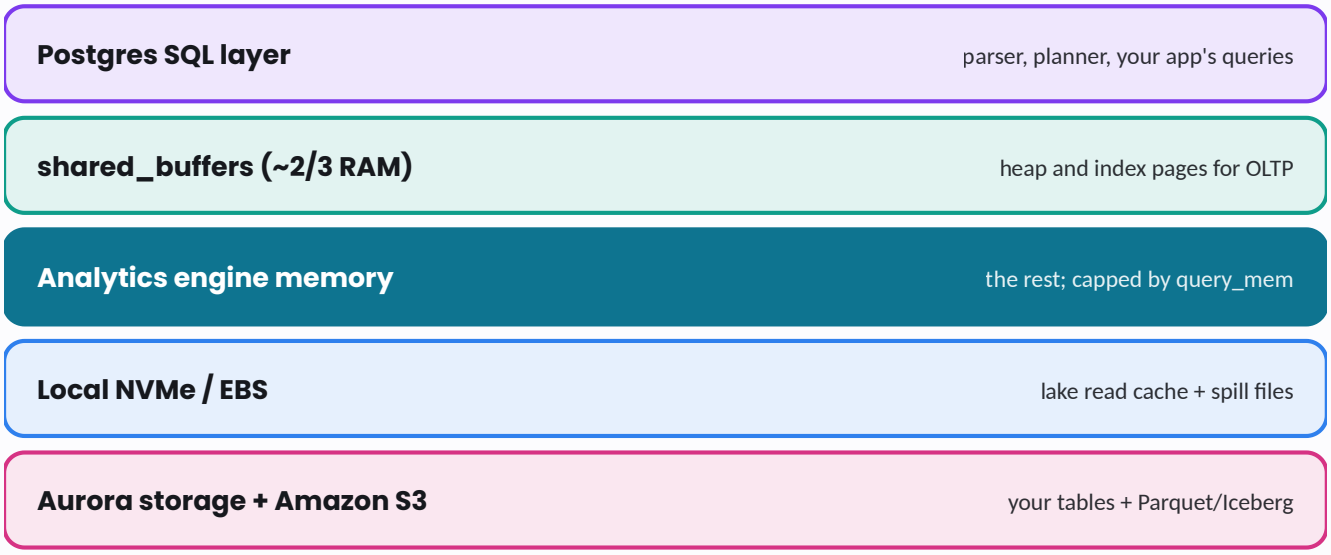


Sharing a box with your OLTP workload

Running a columnar engine inside a database that's also serving your app raises the obvious question: who gets the memory? The resource-management page answers it.[6]

- **Memory.** By default, `shared_buffers` "reserves approximately two-thirds of instance memory", and "Queries involving foreign tables do not use `shared_buffers`". The analytics engine works in the rest: roughly instance memory minus `shared_buffers`. Each query's budget is `aurora_analytics.query_mem`, whose default "is derived from instance memory" and which is capped at that remaining memory.
- **Threads** are managed for you: Aurora "scales it from the query's memory budget", and you don't configure them.
- **Memory pressure.** If memory runs short, Aurora cancels in-progress analytics queries with `ERROR: [Aurora Analytics] Out of Memory Error`. Queries by `rds_superuser` are exempt.
- **Spill and cache share local storage.** Big intermediate results spill to local disk, which is the same NVMe (on `db.r8gd`-type instances with I/O-Optimized) or EBS (on `db.r8g`, `db.r8i`) that holds the read cache. AWS warns that "heavy spilling can create local storage I/O contention that indirectly affects local PostgreSQL operations", and that "there is no fixed formula for the safe number of concurrent queries."

One Aurora instance, two engines



Simplified from the Aurora resource-management docs; proportions are AWS's stated default, not a measurement.

You can watch all of this. `EXPLAIN ANALYZE` adds per-query lines for cache hit bytes, remote read bytes and S3 HEAD/GET counts, and the same figures exist as instance-wide CloudWatch metrics. `aurora_analytics_stat_statements()` reports per-statement rows scanned, bytes read from S3 and cache hits, and `aurora_analytics_cache_size()` shows the cache size.[5,6,1]

WHAT IT MEANS FOR YOU

If you're on Aurora PostgreSQL 17 or 18 and already land history in S3, this is worth trying now: it costs nothing extra to enable, and it can replace a "load last year's data into Postgres just for this report" job. The intended pattern from the announcement is live rows in Aurora, history in the lake, one query across both.[1]

Make `EXPLAIN (VERBOSE)` your habit. The guide's rule of thumb is simple: "A single `Custom Scan` at the top level of the plan = full pushdown (fastest)." Nested `Custom Scan` nodes mean the fallback. Check the `Filters` line under `READ_PARQUET` to confirm your `WHERE` reached the scan, and avoid `SELECT *` on foreign tables.[5] Pettus's version: "One node at the top means DuckDB planned it."[8]

Audit your dimension tables' types and collations. `char(n)` columns, non-C collations, arrays, `vector` and PostGIS types in tables you'll join against the lake can all silently force the fallback.[5] That's an easy schema review to do before you build a dashboard on it.

Keep analytics off the writer. Read queries can run on read replicas, while materialisation statements (`CREATE TABLE AS` , `INSERT ... SELECT` , `MERGE`) run on the writer because they write into Aurora.[1] Given the memory and local-disk sharing described above, routing ad-hoc lake queries to a replica is the safer default, and it's what AWS suggests for the lowest-latency patterns.[15]

If you need DuckDB's full SQL, Aurora isn't that. pg_duckdb on self-managed Postgres (or Azure) gives you `read_parquet()` , DuckDB functions and writing back to the lake; `aurora_analytics` gives you managed, IAM-integrated, read-only foreign tables.[7] Pettus's comparison also covers Snowflake's `pg_lake` , which he says is the only one of the four that promises ACID across heap and Iceberg tables, and pgEdge's ColdFront, which he advises you "Test … Don't deploy … yet".[8]

💥 Watch out for

- **Constraints aren't enforced** on foreign tables. `NOT NULL` , `CHECK` and `DEFAULT` "are stored in metadata for documentation purposes but are not enforced when reading external data".[4]
- `FOR UPDATE` **lies.** Row-locking clauses on foreign tables are accepted and rows come back, but "no row-level lock is taken on the external data".[4]
- **Case-only column names** (`"Foo"` and `"foo"`) in one foreign table are rejected.[4]
- **Cache is lost on restart or failover**, so the first queries after a failover go to S3.[6]
- **Stale reads:** if you overwrite a Parquet file in place, an open connection "can read stale data briefly because it holds a cached file handle"; reconnect to see the new data.[6]
- **Spill can still fail** with `No space left on device` for very large intermediate results.[6]
- **"Uncommitted writes":** the announcement says lake queries can include "uncommitted writes" without defining the phrase.[1] Until AWS documents what that means for visibility across sessions, don't build logic on it.
- **No published benchmarks.** The guide's plan examples are illustrations, not performance claims, and AWS published no comparison with Athena, Redshift or pg_duckdb.[1]

WHAT HAPPENS NEXT

The DuckDB side moves fast. DuckDB's release calendar lists **2.0.0 on 21 October** and 2.0.1 on 18 November, both marked tentative.[16] AWS's announcement says future open-source DuckDB improvements will flow into Aurora and other AWS services, but doesn't say how quickly Aurora picks up new DuckDB versions.[1] AWS's August post named Redshift, Athena, EMR, Glue ETL and SageMaker as integration targets, so expect more "DuckDB inside" launches, perhaps around re:Invent (30 November – 4 December).[14,27]

Open questions worth watching: whether `aurora_analytics` gains write support to Iceberg, whether RDS for PostgreSQL (not Aurora) gets it, and whether AWS publishes the DuckDB version and a compatibility list. And for Postgres itself, the bigger trend is clear: Pettus counted four different projects that plug a lake engine into Postgres, three of them using DuckDB to read Parquet.[8]

📖 Words from this story

Row store: stores each row's columns together; ideal for reading or updating whole records.

Column store: stores each column's values together, so queries read only the columns they need.

Vectorized: processing a batch of values per operation instead of one row at a time.

OLTP: online transaction processing, many small reads and writes of individual records.

OLAP: online analytical processing, fewer, larger scan-and-aggregate queries.

Parquet: an open columnar file format with per-chunk min/max statistics for skipping data.

Apache Iceberg: an open table format tracking which files make up each snapshot of a table.

Catalog: a service mapping table names to their current Iceberg metadata (Glue, REST catalogs).

Foreign data wrapper: a Postgres extension that fetches rows from an external source on demand.

Foreign server: a Postgres object describing one external data source.

Foreign tables: Postgres tables whose data lives elsewhere, read through an FDW.

CustomScan: the Postgres API that lets an extension replace part of the executor; shows as "Custom Scan" in EXPLAIN.

DuckDB: an MIT-licensed, in-process, columnar analytical database.

Dynamic filters: filters built during a join and pushed to the other side's scan.

Bloom filter: a compact set that answers "definitely not" or "maybe", used to discard keys early.

STORY 2 OrioleDB: Postgres without VACUUM, now in public beta

ORIOLEDB BETA18 · SUPABASE · PUBLIC BETA 1 OCT 2026 · ANNOUNCED AT SUPABASE SELECT 2 OCT · ~19 MIN READ

Every Postgres team eventually meets the same ghost. A table that should be 20 GB is 60 GB. Queries slow down for no obvious reason. Somebody discovers that autovacuum has been losing a race for weeks, or that a forgotten idle transaction has been stopping it from cleaning anything. Then one day a warning appears about "transaction ID wraparound", and the on-call engineer learns a new phrase. All of it traces back to a decision Postgres made decades ago about where old versions of rows live. A storage engine called OrioleDB makes the opposite decision, and as of this month you can run it on a paid cloud plan.

WHAT HAPPENED

On **1 October 2026**, Supabase's changelog announced that "OrioleDB is now in Public Beta with paid plans and paid features". You can now create OrioleDB projects "on the Pro, Team, and Enterprise plans, not only Free", and they're "billed the same as standard Postgres projects". New projects run **OrioleDB beta18 on Postgres 17.11**.[17]

OrioleDB projects support compute and disk resizing, pause and restore, Postgres version upgrades, read replicas and scheduled backups. The changelog is just as clear about what's missing:[17]

- "Point-in-Time Recovery (PITR) isn't available for OrioleDB projects."
- "Restoring a backup to a new project isn't available for OrioleDB projects."
- The image lacks `plv8`, `plls`, `plcoffee` and `timescaledb-apache`.
- "We don't recommend OrioleDB for production workloads yet, and it has no SLA."
- You choose OrioleDB when you create a project: "You can't add OrioleDB to an existing project or remove it later."

The next day, **2 October**, Supabase's "Select" conference post put OrioleDB alongside two other launches: Multigres, a Vitess-style "operating system for Postgres" in private alpha, and dbarena, a site for benchmarking managed Postgres providers. The headline claim for OrioleDB: "up to 1.8x higher throughput than Postgres heap in our public, TPC-C-derived benchmarks", with results published on dbarena. The post notes the engine has "been in development since 2021" and has shipped eighteen beta releases.[18] Supabase's OrioleDB docs describe the test as TPC-C at 5,120 warehouses on an 8xlarge instance.[19] Note who's measuring: this is Supabase's engine, benchmarked on Supabase's site. Treat it as a vendor claim.

THE BACKGROUND YOU NEED

Where Postgres keeps old rows

Postgres implements **MVCC** *≈ multi-version concurrency control: instead of locking rows for readers, the database keeps several versions of a row so each transaction sees a consistent snapshot*. Readers never block writers, because each transaction sees the version of each row that was current when its snapshot was taken.

The question is where those versions live. Postgres's answer is: in the table itself. Its default storage, the **heap** ≈ *Postgres's standard table storage: an unordered file of 8 kB pages holding row versions, with indexes pointing to physical locations*, stores each row version as a **tuple** ≈ *a physical row version in Postgres; an UPDATE writes a whole new tuple instead of changing the old one in place*. As the docs have explained for nearly two decades, "an UPDATE or DELETE of a row does not immediately remove the old version of the row". Eventually no transaction needs the old version, and "The space it occupies must be reclaimed for reuse by new rows, to avoid infinite growth of disk space requirements. This is done by running VACUUM."[20]

So every update leaves a **dead tuple** ≈ *an old row version no transaction can see any more; it occupies space until VACUUM reclaims it* behind in the table, and every index entry pointing at it stays too, until **VACUUM** ≈ *Postgres's maintenance command that reclaims dead tuples and freezes old transaction IDs; autovacuum runs it in the background* cleans up. When cleanup can't keep up (a hot table, a long-running transaction holding back the cleanup horizon, an autovacuum that's tuned too gently) you get **bloat** ≈ *space in tables and indexes taken up by dead or reclaimable row versions, making them bigger and slower to scan*.

The 32-bit clock

The second ghost is about counting. Every transaction gets a **transaction ID** ≈ *XID: a number assigned to each writing transaction in Postgres, used to decide which row versions each snapshot can see*, and in the heap these are 32 bits. The docs spell out what that means: "a cluster that runs for a long time (more than 4 billion transactions) would suffer *transaction ID wraparound*: the XID counter wraps around to zero, and all of a sudden transactions that were in the past appear to be in the future — which means their outputs become invisible."[20]

The defence is **freezing** ≈ *marking old row versions as "always in the past" so they stay visible regardless of XID wraparound; done by VACUUM*: VACUUM marks old tuples as permanently in the past, and to stay safe "it is necessary to vacuum every table in every database at least once every two billion transactions".[20] On a busy database, those anti-wraparound vacuums are some of the heaviest maintenance work Postgres does, and they can't be skipped.

The shared buffer hash table

A third, less famous cost is in memory. Postgres caches pages in **shared buffers** ≈ *Postgres's shared page cache in RAM; every page access first looks up the page's location in a shared hash table*, and every time any backend needs a page, it first has to find where (or whether) that page sits in the cache, using a shared lookup table. OrioleDB's authors call this **buffer mapping** ≈ *the step of translating a page's on-disk identity into its location in the shared buffer cache via a shared hash table; a contention point under heavy concurrency* and list it as one of the bottlenecks they set out to remove. In their December 2024 TPC-C runs, they attributed their biggest speedup to exactly this "shared memory cache bottleneck".[21]

Pluggable storage

For most of its history you couldn't replace the heap without forking Postgres. That changed with Postgres 12 (2019), which "ships with a basic infrastructure for table access methods", as Michael Paquier put it at the

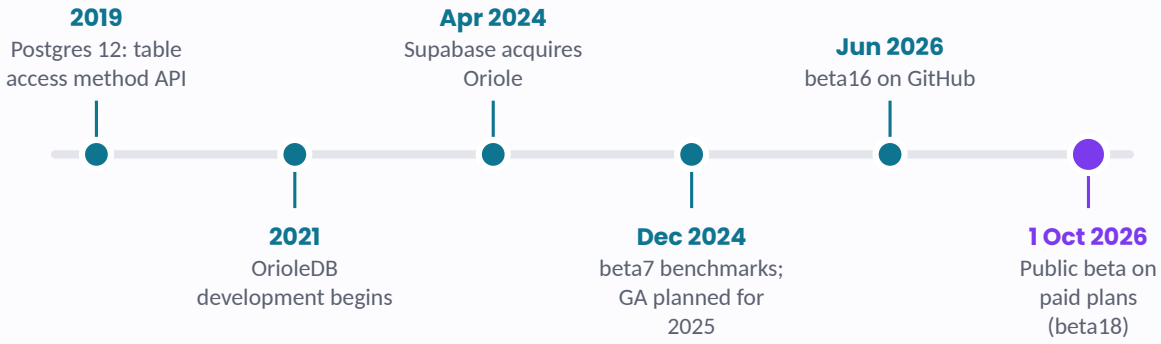
time.[22] The docs describe a **table access method** ≈ *Postgres's pluggable storage API (since version 12): an extension can implement how a table's rows are stored and fetched, selected with CREATE TABLE ... USING* as the interface to code "which manage[s] the storage for tables", noting "it is possible to develop entirely new access method types by writing add-on code".[23]

"Possible" is doing a lot of work there. The table AM API was designed around the heap's assumptions, which is why OrioleDB still needs a patched Postgres to run, as we'll see.

Oriole's long road

OrioleDB was created by Alexander Korotkov, a long-time Postgres contributor.[21] Supabase acquired the Oriole team in April 2024, saying OrioleDB "currently requires a set of patches to Postgres" and that it would work toward upstreaming them, but "We have no timeline for this", adding "it could be a few major Postgres versions away."[24] In December 2024, at beta7, Korotkov and Pavel Borisov published TPC-C results and said they planned general availability in 2025.[21] That didn't happen. The GitHub repository shows beta16 in June 2026, and the cloud service now runs beta18.[25,17]

OrioleDB's road to public beta



Sources: Paquier, Supabase Select, Supabase acquisition, OrioleDB beta7, GitHub, changelog.

HOW IT WORKS

OrioleDB changes four things at once: how rows are organised, where old versions go, how pages are found in memory, and how changes are made durable.

1. The table *is* the primary-key index

In the heap, a table is an unordered pile of pages, and every index (including the primary key) is a separate structure pointing at a physical location. OrioleDB "uses index-organized tables, where table data is stored in the index structure".[19] Its architecture notes are specific: the leaf tuples of the primary index *are* the table rows, and secondary index entries hold their own columns plus the primary key columns, so "the two indexes are linked only by logical values in the rows". If you don't declare a primary key, OrioleDB "automatically creates a primary index on the virtual `ctid` column".[26]

You'll recognise this design if you've used MySQL's InnoDB, which also clusters rows by primary key. The trade-off is familiar too: primary-key lookups and range scans are fast and local, while a secondary-index lookup takes two index descents (secondary, then primary). The OrioleDB authors credit index organisation with better data locality when data doesn't fit in memory.[21]

2. Old versions go to an undo log

This is the change that removes VACUUM. "OrioleDB implements transactions and MVCC using UNDO log."[26] The newest version of a row stays in the data page; older versions are moved into an **undo log** *≈ a separate log holding the previous versions of changed rows, so the table keeps only the latest version and old versions can be discarded once no snapshot needs them*. In the architecture notes' words, "The row-level undo records comprises chains of row versions": the head of each chain sits on the data page and older versions live in undo.[26]

A reader whose snapshot is older than the current version follows the chain back into the undo log until it finds the version it's allowed to see. And once no running transaction can need an old version, that undo space can simply be reused. No scanning the table, no dead tuples left in data pages, no index entries pointing at corpses. Supabase's summary: the undo log "reclaims dead tuples automatically, so bloat and routine VACUUM are no longer needed".[18]

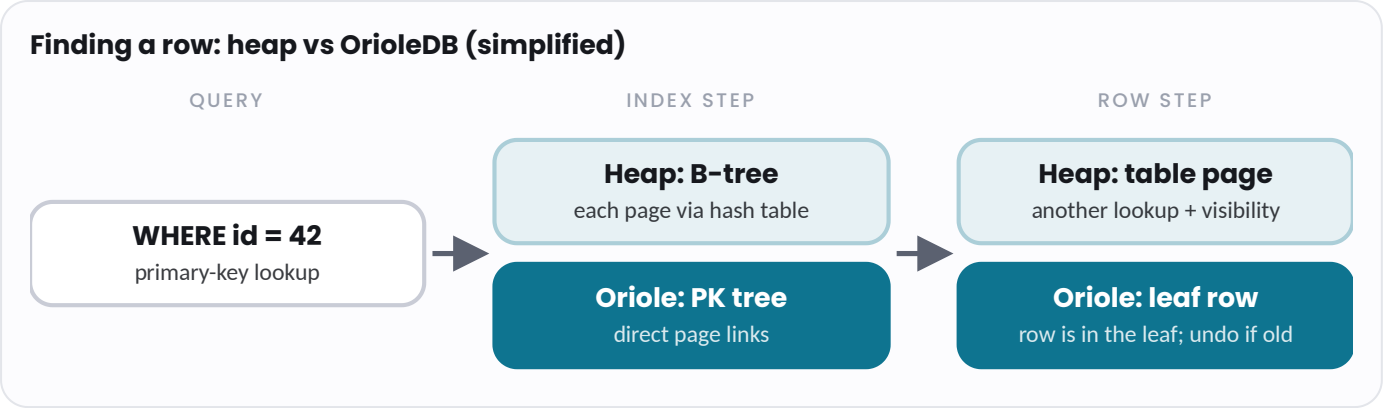
There's a second kind of undo for page structure. OrioleDB has "three types of block-level undo records", for page compaction, splits and merges, each storing images of pages as they were, so a reader with an old snapshot can still see a page's earlier layout.[26]

And the 32-bit clock goes away: OrioleDB uses **64-bit transaction IDs by default**.[18] At a billion transactions a day, a 64-bit counter won't wrap for tens of millions of years, so freezing becomes unnecessary.

🐘 Postgres heap		🐦 OrioleDB
Unordered pages; indexes point at row locations		Rows stored inside the primary-key B-tree
UPDATE leaves a dead tuple in the table		Old versions moved to an undo log
VACUUM reclaims dead tuples and index entries	VS	Undo space reused; no routine VACUUM
32-bit XIDs: freeze before 2 billion		64-bit XIDs by default: no wraparound
Shared buffers found via a hash table		Direct memory/storage links, lock-less reads
WAL logs page changes		Row-level WAL; copy-on-write checkpoints

3. No buffer mapping

Instead of looking each page up in a shared hash table, OrioleDB "uses a so-called `dual pointers` scheme to avoid buffer mapping and corresponding bottlenecks". A link from a parent page in memory can point either at a child page that's already in memory or at its location in storage, so a backend walks the tree without a central lookup. Each page header has a single atomic `state` variable that serves as an exclusive lock and a change counter, so readers can read without locking and simply retry if a writer changed the page underneath them.[26]

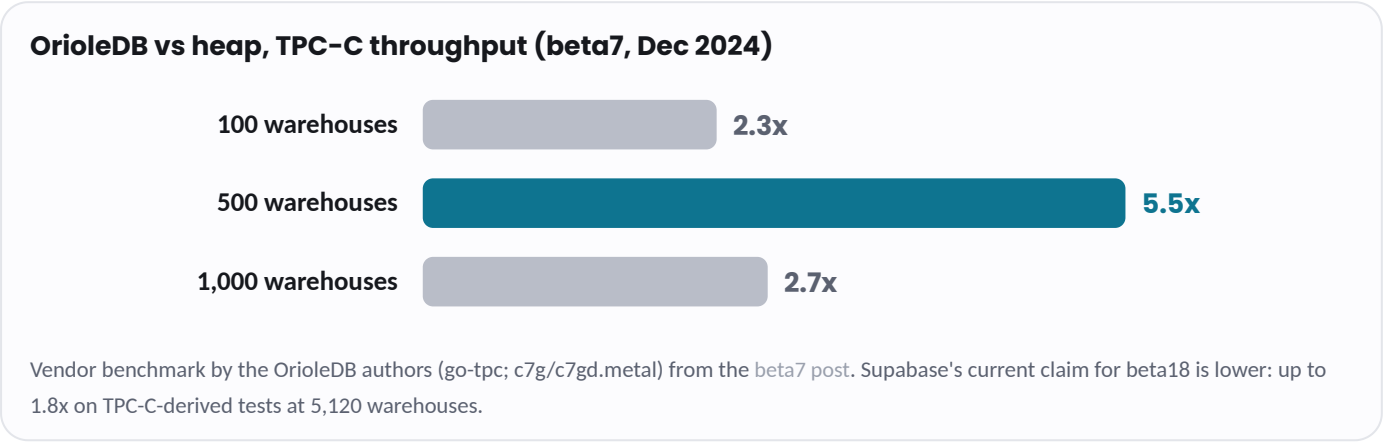


4. Row-level WAL and copy-on-write checkpoints

Durability gets the same treatment. OrioleDB "implements a row-level write-ahead log (WAL) used for both recovery and replication". Only the primary-key and **TOAST** ≈ *The Oversized-Attribute Storage Technique: Postgres's mechanism for storing large values compressed or split into chunks in a side table* trees are logged; secondary indexes are rebuilt from them during recovery, so replay must be idempotent. Recovery runs in parallel across `orioledb.recovery_pool_size` workers.[26]

That works because of **copy-on-write checkpoints** ≈ *checkpoints that write modified pages to free space instead of overwriting them, then update parent pages to point at the new copies, so a consistent tree image always exists on disk* : modified pages are written to new locations, parents are rewritten to point at them, and old images are freed once the checkpoint completes. "Therefore, a consistent tree image exists in storage every moment," so the WAL only needs to describe row changes since that image, not repair half-written pages.[26] The authors credited WAL insertion as the main bottleneck they beat in their smallest TPC-C run.[21]

Here's what the authors measured at beta7, two years ago. These are the team's own numbers, on 64-core Graviton machines with 20 GB of shared buffers:[21]



The gap between 5.5x then and "up to 1.8x" now is worth noticing. The two tests differ in scale, hardware and engine version, so they're not directly comparable, but the newer, larger-scale number is the one Supabase is putting its name to.[18,19]

Using it

On Supabase, you pick "Postgres with OrioleDB" under Postgres Type when creating a project, or run `supabase init --use-orioledb` locally.[17] OrioleDB is then the default for new tables, and you can choose per table with `USING orioledb` or `USING heap`.[19,18] Self-hosted, it's an extension on a patched Postgres:

Self-hosted: heap vs OrioleDB table

Stock Postgres (heap)

```
-- default access method
CREATE TABLE orders (
  id bigint PRIMARY KEY,
  status text,
  total numeric
);
```

Patched Postgres + OrioleDB

```
-- postgresql.conf, then restart:
-- shared_preload_libraries =
--   'orioledb.so'
CREATE EXTENSION orioledb;
CREATE TABLE orders (
  id bigint PRIMARY KEY,
  status text,
  total numeric
) USING orioledb;
```

From the OrioleDB README. Requires PostgreSQL 16 or 17 built from the orioledb/postgres fork with its extensibility patches (or the orioledb/orioledb Docker image).

The README pins specific commits of a Postgres fork "with extensibility patches" for versions 16 and 17, and offers a Docker image, `orioledb/orioledb:latest-pg17`.[25] Supabase's local image moved from "17.9 to 17.11 (OrioleDB patch set 20 to 22)" with this release.[17] OrioleDB is dual-licensed Apache 2.0 or the PostgreSQL License.[25]

WHAT IT MEANS FOR YOU

For most teams, this is a "try it in staging" story, not a migration. Supabase itself says it doesn't recommend OrioleDB for production, there's no SLA, and you can't convert an existing project.[17] What's new is that you can now evaluate it on paid-tier compute, with read replicas and backups, instead of only on the free tier.

The workloads that should care most are the ones where VACUUM is already a fight: tables with very high update rates (counters, queues, session and job-state tables), databases that burn through XIDs fast enough that anti-wraparound vacuums are a recurring event, and high-concurrency OLTP where shared-buffer contention shows up. Those are the bottlenecks OrioleDB's design targets directly.[21,18]

Design for a clustered table. Because rows live in the primary-key tree, your choice of primary key now decides physical order. A random UUID key scatters inserts across the tree; a time-ordered key keeps them together. Secondary indexes carry the primary key, so a wide primary key makes every secondary index bigger.[26] (That's the general behaviour of index-organised tables, familiar from InnoDB; we haven't seen OrioleDB-specific measurements of it.)

Check your extensions and indexes. Supabase says "Native B-tree indexes give the best performance"; other index types built for the heap, such as pgvector's HNSW, run through an "Index Access Method bridge" with

experimental support.[19] OrioleDB tables "support only ICU, C, and POSIX collations".[25] And the extensions missing from Supabase's image are listed above.[17]

Weigh the missing PITR seriously. For many teams, point-in-time recovery is the backstop against "someone ran `DELETE` without a `WHERE`". Scheduled backups exist for OrioleDB projects; PITR doesn't.[17]

> 💥 **Watch out for**
>
> - **It's beta18, not 1.0.** The 2025 GA target set in 2024 slipped, and no new GA date has been announced.[21,18]
> - **Self-hosting means a Postgres fork.** You can't `apt install` it onto stock Postgres; you build or run the patched server, and your Postgres upgrades follow the fork's patch sets.[25]
> - **Benchmarks are the vendor's.** Both the 2024 TPC-C runs and the 1.8x claim come from the OrioleDB/Supabase team, and dbarena is run by Supabase.[21,18]
> - **Known gaps listed at beta7** included unsupported prepared transactions and `REINDEX CONCURRENTLY`.[21] We couldn't find a current, complete limitations list; check the OrioleDB docs for your version before relying on either.
> - **One-way choice on Supabase**: OrioleDB is picked at project creation and can't be added or removed later; existing alpha projects stay on the alpha.[17]

WHAT HAPPENS NEXT

The big question for OrioleDB is the same one Supabase raised in 2024: when, or whether, the patches it needs land in upstream Postgres. Supabase said then it had "no timeline" and that it "could be a few major Postgres versions away".[24] Until they do, OrioleDB is tied to a fork, and that limits it to providers willing to ship one. Supabase hasn't announced a GA date.

Meanwhile, Supabase is building around it. Multigres, from "the team behind Vitess", runs three Postgres nodes across availability zones with "a consensus protocol built on Postgres synchronous replication" and is in private alpha "for testing … not production workloads".[18] A Postgres that doesn't need VACUUM, behind a Postgres that doesn't need a hand-run failover, is clearly the pitch. Whether the community version of Postgres follows is the part to watch.

📖 Words from this story

MVCC: multi-version concurrency control; keeping several row versions so readers get consistent snapshots without locks.

Heap: Postgres's standard table storage, an unordered file of 8 kB pages of row versions.

Tuple: a physical row version; an UPDATE writes a new one.

Dead tuple: an old row version no transaction can see, waiting for VACUUM.

VACUUM: Postgres's maintenance that reclaims dead tuples and freezes old XIDs.

Bloat: wasted space from dead row versions in tables and indexes.

Transaction ID (XID): the number given to each writing transaction, used for visibility; 32 bits in the heap.

Freezing: marking old row versions as permanently "in the past" to survive XID wraparound.

Shared buffers: Postgres's shared page cache in RAM.

Buffer mapping: looking up a page's cache slot in a shared hash table; a contention point.

Table access method: Postgres's pluggable storage API (since 12), chosen with `USING`.

Undo log: a separate log of previous row versions, discarded when no snapshot needs them.

TOAST: Postgres's storage for large values, compressed or chunked into a side table.

Copy-on-write checkpoints: writing changed pages to new locations so a consistent tree is always on disk.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Supabase buys Turso:** alongside a $150 million round led by GIC, Supabase agreed to acquire Turso, maker of a database for spinning up many small isolated databases, saying agents or AI tools create 70% of its new databases.[28]
- **Multigres private alpha:** Supabase's Vitess-team project adds pooling and consensus-based failover across three Postgres nodes in three zones, invite-only and not for production.[18]
- **MySQL Community's thread pool, benchmarked:** Percona measured up to 17.9x more TPS with pooling at 2,560 connections on an I/O-bound sysbench run, but a small slowdown at low connection counts; Percona says MySQL's July releases (26.7.0 and 9.7.2) brought the formerly Enterprise-only thread pool to Community.[29]
- **Flink CDC 3.7.0 (10 Oct):** adds model calls inside CDC pipelines, inline Python UDFs in pipeline YAML, and new SQL Server and Apache Fluss pipeline sources.[30]
- **Snowflake hires Jeff Naughton (12 Oct):** the former UW-Madison professor and Google Distinguished Scientist joins as a Technical Fellow to work on data systems for AI workloads.[31]
- **"Streamhouse" working group:** Aiven, Confluent, Redpanda, StreamNative and Ververica published a vendor-neutral definition of a real-time counterpart to the lakehouse (15 Sep), with the name free for anyone to use.[32]
- **Kafka 4.4.0 still pending:** the Apache Kafka blog's latest post is still the 4.2.2 announcement of 29 September.[33]

1. **Amazon Aurora PostgreSQL now supports direct querying of Apache Iceberg and Parquet data in your data lake**, Esra Kayabali, AWS News Blog, 30 Sep 2026.
2. **Querying Apache Iceberg and Parquet data**, Amazon Aurora User Guide, viewed 13 Oct 2026.
3. **Tutorial: Querying Amazon S3 data**, Amazon Aurora User Guide, viewed 13 Oct 2026.
4. **Working with foreign tables**, Amazon Aurora User Guide, viewed 13 Oct 2026.
5. **Execution plan**, Amazon Aurora User Guide, viewed 13 Oct 2026.
6. **Resource management**, Amazon Aurora User Guide, viewed 13 Oct 2026.

7. **Amazon Aurora's analytics is gated DuckDB: comparison with pg_duckdb**, Franck Pachot, DEV Community, 5 Oct 2026 (edited 10 Oct).
8. **The International Ice Patrol**, Christophe Pettus (PGX Inc.), The Build, 8 Oct 2026.
9. **Foreign Data**, PostgreSQL 18 documentation.
10. **CustomScanAPI**, PostgreSQL wiki.
11. **DuckLabs to Join AWS, Projects to Remain Open Source**, Mark Raasveldt and Hannes Mühleisen, DuckDB, 26 Aug 2026.
12. **Announcing pg_duckdb Version 1.0**, Jacob Matson and Jelte Fennema-Nio, MotherDuck, 3 Sep 2025.
13. **Amazon to acquire DuckLabs, adding the team behind DuckDB amid broader shakeup in cloud data**, GeekWire, 26 Aug 2026.
14. **AWS and DuckLabs: Building the future of analytics together**, Mai-Lan Tomsen Bukovec, AWS Big Data Blog, 26 Aug 2026.
15. **AWS enables Aurora to analyze data directly from S3**, Techzine, 1 Oct 2026.
16. **Release Calendar**, DuckDB, viewed 13 Oct 2026.
17. **OrioleDB is now in Public Beta with paid plans and paid features**, Supabase changelog, 1 Oct 2026.
18. **Scale without limits: Multigres, OrioleDB, and dbarena**, Joe Sciarrino, Alexander Korotkov, Daniel Mitterdorfer and Aditya Maruvada, Supabase, 2 Oct 2026.
19. **OrioleDB Overview**, Supabase documentation, viewed 13 Oct 2026.
20. **Routine Vacuuming**, PostgreSQL 8.3 documentation (the dead-tuple and 32-bit XID explanation is unchanged in principle in current versions).
21. **OrioleDB beta7: Benchmarks**, Alexander Korotkov and Pavel Borisov, OrioleDB, 1 Dec 2024.
22. **Postgres 12 highlight: Table Access Methods and blackholes**, Michael Paquier, 4 Jun 2019.
23. **Table Access Method Interface Definition**, PostgreSQL documentation.
24. **Supabase acquires OrioleDB**, Paul Copplestone, Supabase, 15 Apr 2024.
25. **orioledb/orioledb**, GitHub repository and README, viewed 13 Oct 2026.
26. **OrioleDB architecture overview**, OrioleDB documentation on GitHub, viewed 13 Oct 2026.
27. **Village News: MySQL News + Events (6 October 2026)**, VillageSQL, 6 Oct 2026 (events list).
28. **Supabase raises $150M and agrees to acquire Turso for agent databases**, RuntimeWire, 2 Oct 2026.
29. **Thread Pool in Percona Server and MySQL (Part 1)**, Bogdan Degtyariov, Percona, 5 Oct 2026.
30. **Apache Flink CDC 3.7.0 Release Announcement**, Apache Flink, 10 Oct 2026.
31. **Snowflake Adds Database Veteran Jeff Naughton to Engineering Ranks**, BigDATAwire, 12 Oct 2026.
32. **Aiven, Confluent, Redpanda, StreamNative and Ververica Form Streamhouse Working Group**, Streamhouse Working Group, Confluent blog, 15 Sep 2026.
33. **Apache Kafka blog**, Apache Kafka, viewed 13 Oct 2026.